

FC7xxx SENT Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/12/15	Initial release

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area.....	9
Chapter 5 Interrupt Service Routine (ISR).....	10
Chapter 6 Error Report.....	11
6.1 Det.....	11
6.2 Dem.....	11
Chapter 7 Function Calls.....	12
7.1 Function Calls during Startup	12
7.2 Function Calls during Shutdown	12
7.3 Function Calls during Wake-up	12
7.4 Function Calls during Runtime	12
Chapter 8 Other Requirements	13
8.1 Notification, Callback, Callout	13
8.2 Macros	13
Chapter 9 Integration Steps.....	14

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the SENT module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for SENT module
- Port: This module provides the port configurations if want output SENT signal through pin.
- Common: This module is the basic module which used to choose the chip.
- OS: This is used for SENT main function reference.
- EcuM: is required for selecting the reference to the wakeup source for every SENT controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the SENT module, the user needs to initialize the MCU module first.
- Port: To output SENT signal, the user needs to initialize PORT module first and assign the UART pins.

2.2 Files Required for Compile

SENT module files:

- _MCAL/Src/Sent/Src/CDD_Sent.c
- _MCAL/Src/Sent/Src/Sent_Hal.c
- _MCAL/Src/Sent/Src/Sent_HWA.c
- _MCAL/Src/Sent/Src/Sent_Isr.c
- _MCAL/Src/Sent/include/CDD_Sent.h
- _MCAL/Src/Sent/include/Sent_Hal.h
- _MCAL/Src/Sent/include/Sent_HWA.h
- _MCAL/Src/Sent/include/Sent_MemMap.h
- _MCAL/Src/Sent/include/Sent_Reg.h
- _MCAL_generate/src/CDD_Sent_cfg.c
- _MCAL_generate/include CDD_Sent_cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h

- `_MCAL/Src/Common/include/CompilerDefinition.h`

Det module files:

- `Det.h`

Rte module files:

- `SchM_Sent.h`

2.3 Add Plug-ins

SENT module plug-ins are developed for EB tresos Studio, so, to use SENT plug-ins on the EB tresos Studio, the user needs to add the SENT module to the EB plug-ins folder first.

- 1) Copy the SENT module (`_MCAL/EB_Plugins/eclipse/plugins/SENT`) folder to EB tresos plug-ins (`EB/tresos/plugins/`) folder.
- 2) Set the SENT module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the SENT module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
SENT_START_SEC_CONFIG_DATA_8 SENT_STOP_SEC_CONFIG_DATA_8 SENT_START_SEC_CONFIG_DATA_16 SENT_STOP_SEC_CONFIG_DATA_16 SENT_START_SEC_CONFIG_DATA_32 SENT_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).
SENT_START_SEC_CONFIG_DATA_UNSPECIFIED SENT_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
SENT_START_SEC_CONST_BOOLEAN SENT_STOP_SEC_CONST_BOOLEAN SENT_START_SEC_CONST_8 SENT_STOP_SEC_CONST_8 SENT_START_SEC_CONST_16 SENT_STOP_SEC_CONST_16 SENT_START_SEC_CONST_32 SENT_STOP_SEC_CONST_32 SENT_START_SEC_CONST_UNSPECIFIED SENT_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
SENT_START_SEC_CODE SENT_STOP_SEC_CODE SENT_START_SEC_RAMCODE SENT_STOP_SEC_RAMCODE SENT_START_SEC_CODE_AC SENT_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
SENT_START_SEC_VAR_NO_INIT_BOOLEAN SENT_STOP_SEC_VAR_NO_INIT_BOOLEAN SENT_START_SEC_VAR_NO_INIT_8 SENT_STOP_SEC_VAR_NO_INIT_8 SENT_START_SEC_VAR_NO_INIT_16 SENT_STOP_SEC_VAR_NO_INIT_16 SENT_START_SEC_VAR_NO_INIT_32 SENT_STOP_SEC_VAR_NO_INIT_32 SENT_START_SEC_VAR_NO_INIT_UNSPECIFIED SENT_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
SENT_START_SEC_VAR_INIT_BOOLEAN SENT_STOP_SEC_VAR_INIT_BOOLEAN SENT_START_SEC_VAR_INIT_8 SENT_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
SENT_START_SEC_VAR_INIT_16		startup code (data).
SENT_STOP_SEC_VAR_INIT_16		
SENT_START_SEC_VAR_INIT_32		
SENT_STOP_SEC_VAR_INIT_32		
SENT_START_SEC_VAR_INIT_UNSPECIFIED		
SENT_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Exclusive Area

SENT module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the SENT driver:

- SENT_Hal.c:
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_00 is used in Sent_HL_Instance_Init.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_01 is used in Sent_HL_Instance_Init.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_02 is used in. Sent_ML_Channel_Init
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_03 is used in Sent_ML_Channel_Init.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_04 is used in Sent_ML_Channel_Init.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_05 is used in Sent_LL_SPC_Init.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_06 is used in Sent_LL_Channel_DeInit.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_07 is used in SENT_Hal_RequestSPCPulse.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_08 is used in SENT_Hal_RequestSPCPulse.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_09 is used in Sent_HL_GetFastMessage.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_10 is used in Sent_HL_GetFastMessage.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_11 is used in Sent_HL_GetSlowMessage.
 - SchM_Enter_Sent_SENT_EXCLUSIVE_AREA_12 is used in Sent_HL_GetSlowMessage.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
SENT0	SENT_IsrSENT0_All	144
SENT1	SENT_IsrSENT1_All	145

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Sent_Init	SENT_INITIALIZED; SENT_E_INIT_FAILED_U8; SENT_E_PARTITION_MAPPING; SENT_E_PARAM_U8;
Sent_GetVersionInfo	SENT_E_PARAM_U8;
Sent_DeInit	SENT_E_PARTITION_MAPPING ; SENT_E_UNINIT_U8;
Sent_RequestSpcPulse	SENT_E_UNINIT_U8;
Sent_MainFunctionFastMessageRead	SENT_E_UNINIT_U8;
Sent_MainFunctionSlowMessageRead	SENT_E_UNINIT_U8;

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

SENT shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is SENT_Init(). The MCU module should be initialized before the SENT is initialized. The SENT module shall be initialized by SENT_Init(<&SENT_Configuration>) service call during the start-up before the SENT peripherals are used. Please note that GPIO pins used for connection of SENT physical layer have to be properly assigned to the desired module prior to the SENT initialization: so the MCU and PORT modules shall be initialized before SENT is initialized. After the SENT module is initialized, each SENT controller has to be initialized as well before using it. This is also done by the SENT_Init(<&SENT_Configuration>) service.

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

Users should call Sent_MainFunctionFastMessageRead and Sent_MainFunctionSlowMessageRead in period to get the sent message to avoid to lost the message. They should be call while FastMsgReadType and SlowMsgReadType are configured as Polling in EB Tresos.

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification
There are no callback functions within the SENT driver.
- Callback
There are no callback functions within the SENT driver.
- Callout
The SENT Driver uses these mandatory callouts functions which have to be provided by the respective SENT stack:
SentFastNotification and SentSlowNotification.
These functions are optional:
SentSlowErrorNotification and SentFastErrorNotification.

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:
AUTOSAR_OS_NOT_USED needs to be defined.
- If OS is used:
Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the SENT module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the SENT module with dependent modules.